

Encapsulation and Data Protection in Python

Track: Data Science | Level 1 - Day 16 | Tools: Python 3, Jupyter Notebook

Level 1 - Day 16: Encapsulation and Data Protection in Python

Track: Data Science

Task: Task 16 - Encapsulation and Information Hiding

Objective: Master public, protected (`_var`), and private (`__var`) access modifiers, getter/setter patterns, the `@property` decorator, and protecting sensitive machine learning states.

1. Quick Reference & Core Concepts

Access Modifier	Naming Convention	Intended Usage
Public	<code>self.name</code>	Freely accessible from anywhere
Protected	<code>self._model_type</code>	Internal module/subclass convention (hint)
Private	<code>self.__api_key</code>	Name mangling to prevent outside modification
@property Getter	<code>@property def balance(self):</code>	Clean read-only access to managed attributes
@setter Decorator	<code>@balance.setter def balance(self, val):</code>	Input validation and quality constraint checks

2. Practical Code Examples

In [code]

```
# Example 1: Public, Protected, and Private Attributes
class SecureAccount:
    def __init__(self, username, balance, ssn):
        self.username = username      # Public
        self._balance = balance       # Protected
        self.__ssn = ssn              # Private

    def get_masked_ssn(self):
        return f"***-**-{str(self.__ssn)[-4:]}"

acc = SecureAccount("neha_analytics", 45000, 123456789)
print(f"Ex 1 - User: {acc.username} | Protected Balance: ${acc._balance:,} | SSN: {acc.get_masked_ssn()}")
# Attempting direct private access raises AttributeError
try:
    print(acc.__ssn)
except AttributeError as e:
    print(f"Ex 1 - Private attribute protected: {e}")
```

In [code]

```
# Example 2: The @property Decorator with Setter Validation
class TemperatureSensor:
    def __init__(self, celsius):
        self._celsius = None
        self.celsius = celsius # Invokes setter

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Temperature below absolute zero is impossible!")
        self._celsius = value

sensor = TemperatureSensor(25.0)
print(f"Ex 2 - Valid Temperature: {sensor.celsius}°C")
try:
    sensor.celsius = -300
except ValueError as err:
    print(f"Ex 2 - Setter Blocked Invalid Input: {err}")
```

3. Applied Data Science Pipeline

Applied Data Pipeline: Encapsulated Model Hyperparameter Registry

In [code]

```
class HyperparameterRegistry:
    def __init__(self, learning_rate, max_depth):
        self._learning_rate = None
        self._max_depth = None
        self.learning_rate = learning_rate
        self.max_depth = max_depth

    @property
    def learning_rate(self):
        return self._learning_rate

    @learning_rate.setter
    def learning_rate(self, lr):
        if not (0.0 < lr <= 1.0):
            raise ValueError(f"Learning rate must be in (0, 1], got {lr}")
        self._learning_rate = lr

model_config = HyperparameterRegistry(0.05, 6)
print(f"Active Learning Rate: {model_config.learning_rate}")
```